EdgeNav-X

STM32 Cortex-M4 Autonomous Perception and Motion Control Platform

Technical Project Report

Submitted By

ANORA SHARON TESSIE S

Register Number: 3122233002011

Final Year B.E. Electronics and Communication Engineering

Department of Electronics and Communication Engineering

SSN College of Engineering

Kalavakkam, Chennai, Tamil Nadu

Contents

1 Introduction

Autonomous mobile robots are increasingly used in industrial automation, warehouse logistics, and smart transportation systems. These robots must be capable of sensing their environment, making decisions, and controlling their movement without human intervention. Achieving reliable autonomous navigation requires the integration of sensing, processing, and motion control within a real-time embedded platform.

This project, **EdgeNav-X**, presents the development of an STM32 Cortex-M4 based Autonomous Perception and Motion Control Platform. The system is built around the STM32F446RE microcontroller and integrates an IR obstacle detection sensor, a TCS34725 RGB color sensor, an L298N motor driver, and TT geared DC motors to enable autonomous navigation.

The robot continuously monitors its surroundings for obstacles. Upon detecting an obstacle, it stops, identifies the detected color, and performs a predefined navigation action such as turning left, turning right, stopping temporarily, or executing a U-turn. Motor speed control is achieved using hardware PWM generated through independent timer peripherals, enabling stable and accurate movement.

During development, several engineering challenges such as motor drift, stopping accuracy, turn calibration, and color detection under varying lighting conditions were encountered and resolved through hardware calibration and software optimization techniques.

The project demonstrates the practical implementation of embedded systems, sensor interfacing, real-time control, and autonomous robotics using an ARM Cortex-M4 microcontroller platform.

2 Literature Review

The development of autonomous mobile robots requires the integration of sensing, processing, and motion control subsystems within a real-time embedded platform. Modern robotic systems commonly employ microcontrollers, obstacle detection sensors, motor drivers, and intelligent control algorithms to achieve reliable navigation in dynamic environments.

The STM32 family of ARM Cortex-M microcontrollers has gained significant popularity in embedded robotics due to its high processing capability, hardware timers, communication peripherals, and low power consumption. The STM32 Hardware Abstraction Layer (HAL) framework further simplifies peripheral configuration and enables rapid development of real-time embedded applications.

Obstacle detection is a fundamental requirement for autonomous navigation. Infrared (IR) sensors are widely used because of their fast response time, low cost, and simple interfacing requirements. These sensors enable robots to identify nearby obstacles and execute corrective navigation actions to prevent collisions.

Color perception is another important feature used in industrial automation and robotic decision-making systems. The TCS34725 RGB color sensor provides accurate color measurement through dedicated red, green, blue, and clear light channels, allowing embedded platforms

to classify objects based on color information and perform predefined actions accordingly.

Motion control in differential-drive robots is typically achieved through Pulse Width Modulation (PWM). Research indicates that independent speed control of left and right motors improves trajectory stability and compensates for mechanical variations between motors. Utilizing separate hardware timers for each motor enables precise duty-cycle adjustment and enhances straight-line navigation performance.

Based on these concepts, EdgeNav-X integrates STM32-based processing, IR obstacle detection, RGB color perception, and independent dual-motor PWM control to create a reliable autonomous perception and motion control platform capable of real-time decision making and navigation.

3 Components Required

The autonomous perception and motion control platform is built using specialized hardware components selected for reliable sensing, deterministic motion control, and efficient power management. Table 1 summarizes the hardware used and their respective functions within the EdgeNav-X architecture.

Table 1: EdgeNav-X Hardware Component Specifications

Component Name	**Purpose / Technical Function**
STM32F446RE Nucleo	ARM Cortex-M4 microcontroller responsible for sensor acquisition, navigation logic, PWM generation, and real-time system control.
L298N Dual H-Bridge Driver	Interfaces the low-power STM32 control signals with the DC motors, enabling bidirectional motor operation and speed control.
TCS34725 RGB Color Sensor	I^2C-based color sensing module used for real-time detection and classification of colored objects.
IR Obstacle Detection Sensor	Infrared proximity sensing module used to detect nearby obstacles and trigger navigation decisions.
TT Geared DC Motors	Differential drive actuators providing robot locomotion through independent left and right wheel control.
Buck Converter Module	High-efficiency DC-DC step-down regulator used to convert battery voltage to stable operating voltages for electronic subsystems.
Toggle Switch	Primary power isolation device used for safe system startup and shutdown control.
Lithium-Ion Battery Pack	Portable power source supplying energy to the motor driver, sensors, and microcontroller platform.
USART2 Debug Interface	Serial communication channel configured for runtime diagnostics, debugging, and telemetry monitoring.

4 System Architecture

The EdgeNav-X platform is designed as an integrated perception and motion control system centered around the STM32F446RE ARM Cortex-M4 microcontroller. The architecture combines sensing, decision-making, motor control, and power management subsystems to achieve autonomous navigation.

The IR obstacle detection sensor continuously monitors the environment for nearby obstacles and provides digital feedback to the STM32. Upon obstacle detection, the microcontroller immediately halts the robot and initiates the perception sequence. The TCS34725 RGB color sensor then acquires color information through the I^2C communication interface and transmits raw Red, Green, Blue, and Clear channel data to the STM32 for processing.

The embedded firmware executes color normalization and multi-sample voting algorithms to improve classification reliability under varying lighting conditions. Based on the identified color, the navigation decision engine determines the appropriate action, such as turning left, turning right, stopping temporarily, or performing a U-turn.

Motion control is achieved through an L298N dual H-bridge motor driver interfaced with two TT geared DC motors arranged in a differential drive configuration. Independent PWM signals generated using TIM2 and TIM3 handles allow precise speed control of the left and right motors, improving trajectory stability and reducing directional drift.

The entire system is powered by a rechargeable battery pack, while a DC-DC buck converter provides regulated voltage levels required by the microcontroller and sensor modules. This architecture enables reliable autonomous operation through the seamless integration of sensing, processing, decision-making, and actuation subsystems.

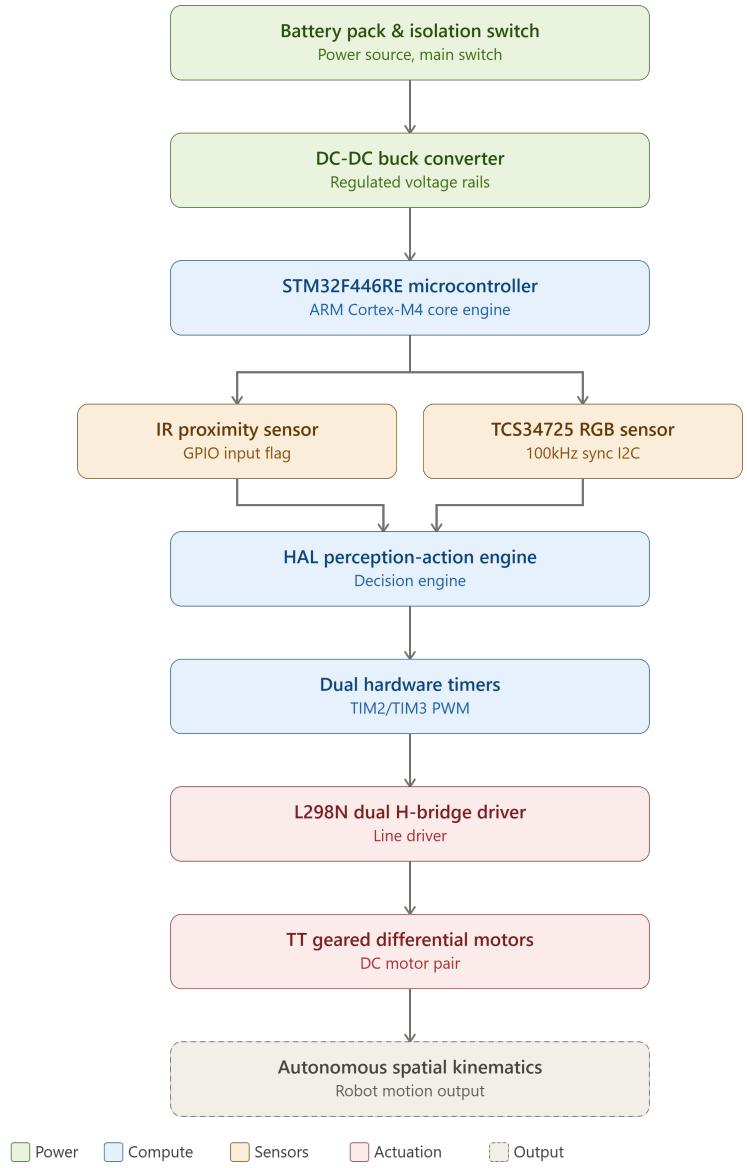

Figure 1: Overall System Microarchitectural & Data Flow Topology of EdgeNav-X

The complete EdgeNav-X hardware architecture consists of the STM32F446RE microcontroller, TCS34725 RGB color sensor, IR obstacle detection sensor, L298N motor driver, TT geared DC motors, buck converter, battery pack, and toggle switch integrated into a unified autonomous navigation platform. The physical implementation, component distribution, and structural alignment are detailed through multiple orthogonal views of the prototype assembly.

4.1 Complete Hardware Connections

The complete EdgeNav-X hardware architecture consists of the STM32F446RE microcontroller, TCS34725 RGB color sensor, IR obstacle detection sensor, L298N motor driver, TT geared DC motors, buck converter, battery pack, and toggle switch integrated into a unified autonomous navigation platform. The physical implementation, component distribution, and structural alignment are detailed through multiple orthogonal views of the prototype assembly.

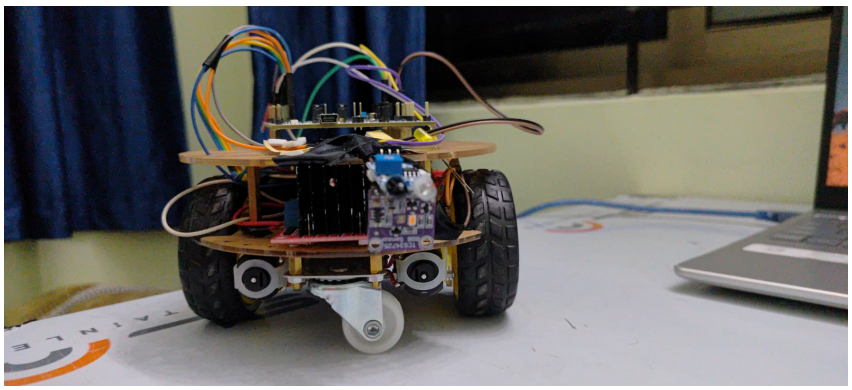

Figure 2: Front View of the EdgeNav-X Autonomous Navigation Robot

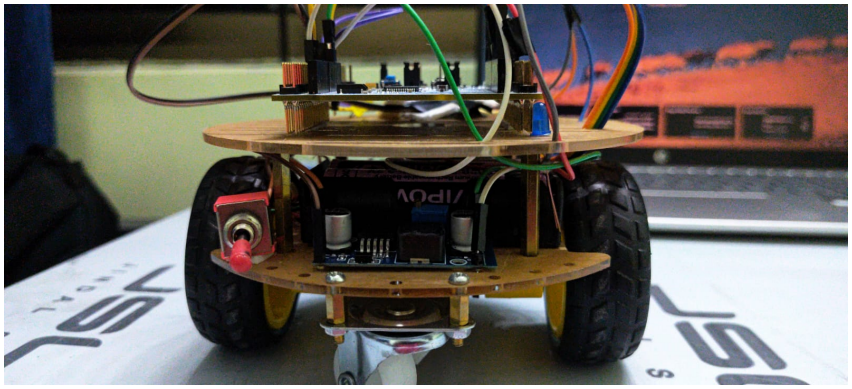

Figure 3: Rear View Showing Power System and Drive Assembly

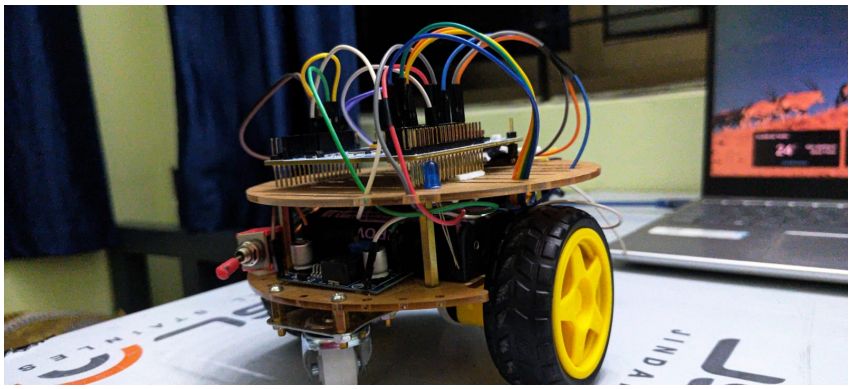

Figure 4: Integrated Hardware Assembly of EdgeNav-X

The STM32F446RE serves as the central processing unit and interfaces with all peripherals. The TCS34725 RGB sensor communicates through the I^2C bus for color recognition, while the IR sensor provides real-time obstacle detection through a dedicated GPIO input. Independent PWM channels generated using TIM2 and TIM3 are routed to the L298N motor driver, enabling precise control of the left and right TT geared DC motors.

Power is supplied through a rechargeable battery pack connected through a master toggle switch. A buck converter regulates the battery voltage and provides a stable supply rail for the STM32 controller and sensors. All modules share a common ground reference to ensure reliable signal integrity and minimize electrical noise during operation.

5 Software Design

The software architecture of EdgeNav-X is structured to manage low-level micro-architectural processes and higher-level navigation routines. The firmware is developed in Embedded C to achieve deterministic, real-time responses with minimal execution latency by directly manipulating hardware registers and utilising the STM32 Hardware Abstraction Layer (HAL).

5.1 System Initialization

The system initialization sequence configures the internal hardware peripherals of the STM32F446RE microcontroller immediately following a power-on reset (POR). The firmware execution flow prioritizes hardware stabilization and communications bus health before entering the primary infinite control loop.

5.1.1 System Clock Configuration

The system clock is initialized utilizing the internal High-Speed Internal (HSI) oscillator source. As structured in the `SystemClock_Config()` routine, the Phase-Locked Loop (PLL) is bypassed (`RCC_PLL_NONE`) and the HSI signal is routed directly as the main system clock source (`RCC_SYSCLKSOURCE_HSI`). Consequently, the core processing clock frequency (SYSCLK) operates at a stable baseline of 16 MHz. The High-Speed APB bus (APB2) and Low-Speed APB bus (APB1) prescalers are configured to a division factor of 1 (`RCC_SYSCLK_DIV1`), ensuring peripheral clocks run flush with the primary oscillator frequency.

5.1.2 Peripheral and GPIO Configuration

Following clock configuration, the core abstraction layers are mounted via `HAL_Init()`. Specific hardware peripheral blocks are then locked into their operational modes:

- **GPIO Initialization:** Advanced pin mapping registers handle input/output routing. The infrastructure configures `GPIOA_PIN_8` as a digital input with an internal weak pull-up resistor to monitor the IR obstacle sensor flag. Directional control pins for the H-bridge (`GPIOA_PIN_10` for `IN1`; `GPIOB_PIN_3`, `PIN_4`, and `PIN_5` for `IN2`, `IN4`, and `IN3` respectively) are mapped as low-speed, push-pull digital outputs.

- **I2C1 Communication Bus:** The `I2C1` peripheral instance is initialized via `MX_I2C1_Init()` to run in standard mode at a 100 kHz clock speed. It enforces 7-bit addressing, clock stretching capability, and disables general-call or dual-addressing modes to guarantee error-free synchronous tracking with the TCS34725 sensor.
- **TIM2 and TIM3 Hardware Timers:** Hardware timer engines configure specialized PWM generation channels. Timer 3 Channel 2 (`TIM3_CH2` mapped to Alternate Function `GPIO_AF2_TIM3` on `PC7`) commands the Left motor speed line (`D9`), while Timer 2 Channel 3 (`TIM2_CH3` mapped to Alternate Function `GPIO_AF1_TIM2` on `PB10`) commands the Right motor speed line (`D11`). Both counters use a prescaler of 83 and an auto-reload period (ARR) of 999 to establish a switching frequency of 192 Hz for smooth, discrete motor speed variations.
- **USART2 Debugging Port:** Operating at 115200 Baud with an 8-bit word length, no parity, and 1 stop bit, this asynchronous serial block channels runtime parameters out to an external host terminal via a custom `Debug_Print()` wrapper function.

5.2 Motor Control Algorithm

Chassis locomotion, trajectory alignment, and steering profiles are governed by hardware-timed Pulse Width Modulation (PWM) channels running on a 16-bit counter register framework. The firmware controls differential kinematics by adjusting the duty cycle compare registers assigned to the left and right wheels.

5.2.1 PWM Duty Cycle Control and Inertia Override

The velocity optimization layer directly alters the fractional timer compare registers (Capture/-Compare Registers). To overcome physical static friction and rotor inertia during transitions from a dead stop, the firmware implements an aggressive kickstart pulse technique. When commanding forward travel via `Robot_Forward()`, the compare registers are momentarily pulsed at a heavy duty cycle (`CCR = 950`) for 25 ms before dropping to a stable, low-speed cruise duty cycle of 450.

5.2.2 Directional Logic Control

Steering vectors are executed by directly modifying the digital states of the microcontroller GPIO lines wired to the L298N driver inputs (`IN1` through `IN4`). Changing these logic combinations dictates forward, reverse, pivot turns, and stop maneuvers by setting matching high (`GPIO_PIN_SET`) or low (`GPIO_PIN_RESET`) voltage states across the control pins.

5.3 Color Detection Algorithm

The color classification engine monitors surface markings to track positional updates and trigger directional shifts. Raw chromatic data collection is managed via synchronous peripheral memory reads over the shared interface bus.

5.3.1 Chromatic Data Ingestion

The software communicates with the TCS34725 sensor using the `Read_Color_Values()` function. This routine performs sequential, 8-bit memory-mapped pointer lookups via `HAL_I2C_Mem_Read()`, reading two consecutive bytes starting at the device's command-masked data registers for Clear (`0x14`), Red (`0x16`), Green (`0x18`), and Blue (`0x1A`). The resulting byte pairs are bit-shifted and combined into dedicated 16-bit volatile variables (`Clear_Color`, `Red_Color`, `Green_Color`, and `Blue_Color`).

5.3.2 Normalization and Classification Rules

To isolate the true color of the navigation path from variations in ambient light and shadows under the chassis, the raw 16-bit color intensity readings are normalized down to an 8-bit proportional scale relative to the Clear channel:

$$r_{scaled} = \frac{\text{Red\_Color} \times 255}{\text{Clear\_Color}}, \quad g_{scaled} = \frac{\text{Green\_Color} \times 255}{\text{Clear\_Color}}, \quad b_{scaled} = \frac{\text{Blue\_Color} \times 255}{\text{Clear\_Color}}$$

The normalized values are clamped to a maximum 8-bit scale factor of 255. The classification engine then processes these ratios using conditional logic boundaries to determine the path color.

5.4 Obstacle Detection Algorithm

The obstacle detection subsystem utilizes an edge-triggered hardware polling mechanism to monitor the environment ahead of the chassis. The implementation balances rapid hardware response times with software-defined stabilization windows to prevent false-positive triggers from vibration or optical noise.

5.4.1 Hardware Debouncing Protocol

The active-low digital flag from the IR proximity sensor is tied to a dedicated general-purpose input pin (`GPIOA_PIN_8`) configured with internal weak pull-up resistors. When an object triggers the sensor and pulls the line down to logic state `0`, the firmware initiates a hardcoded 40 ms software debounce delay window using `HAL_Delay(40)`. Following this window, a second discrete read is executed. If the input pin has returned to a high state, the event is dropped as a transient noise spike, and the primary execution loop resumes without altering motor states.

5.4.2 Active Braking and Mechanical Settling Windows

Upon verification of a valid obstacle flag, the processor halts the robot using a multi-phase active braking sequence inside the `Robot_Stop()` function. The microcontroller manipulates the GPIO pins linked to the H-bridge directional inputs (`IN1` through `IN4`) to apply a reverse voltage across both motor windings, generating instantaneous counter-torque.

To eliminate physical forward drift, asymmetric duty cycles are applied via the timer capture/compare registers (`htim3.Instance->CCR2` at 850 for the left motor; `htim2.Instance->CCR3` at 750 for the right motor) for a precise duration of 40 ms. Once forward momentum is broken, the PWM signals are set to 0, and all directional bridge pins are driven low to isolate the coils. The firmware then executes an unblocked 2-second settling hold delay (`HAL_Delay(2000)`) to eliminate physical chassis sway and mechanical shake before starting any optical readings.

5.5 Navigation Decision Logic

Once the chassis settles completely, the navigation framework transitions into a data-acquisition state to evaluate the driving surface and calculate steering vectors.

5.5.1 Multi-Sample Voting Topology

To ensure high classification accuracy under challenging, low-lux conditions, the decision engine implements a 5-sample majority voting architecture. The firmware loops five consecutive times, querying the raw Clear, Red, Green, and Blue sensor registers of the TCS34725 over the `I2C1` bus via `Read_Color_Values()`. Each data collection cycle is separated by a 110 ms integration delay.

Independent local counters track votes for individual color bands. For standard target classification, a target color wins the sample point if its scaled metric is strictly dominant over the alternative channels (e.g., $g_{scaled} > r_{scaled}$ and $g_{scaled} > b_{scaled}$ registers for a Green track token). A target must accumulate a minimum threshold of 3 out of 5 valid votes to declare an execution lock.

5.5.2 State Mapping and Kinematic Profiles

The system maps victorious color locks directly onto specialized navigational actions, which are processed alongside a 3-second diagnostic delay window to allow visual telemetry validation over the serial debugging port:

- **White (`white_votes` $\geq$ 3):** Triggers `Robot_UTurn()`. The H-bridge inputs reverse directional logic, driving both channels at a heavy duty cycle of 980 for 30 ms to break inertia, settling into a controlled 460 duty cycle spin for 750 ms.
- **Red (`red_votes` $\geq$ 3):** Calls an immediate `Robot_Stop()` sequence and locks the motor brakes for a 5-second hold interval before resuming path sweeps.
- **Green (`green_votes` $\geq$ 3):** Coordinates a right-hand pivot maneuver via `Robot_Turn_Right()` by establishing a differential wheel rotation profile running at a 160 duty cycle for an increased duration of 410 ms to eliminate under-turn drift.
- **Blue (`blue_votes` $\geq$ 3):** Coordinates a left-hand pivot maneuver via `Robot_Turn_Left()` running at a 160 duty cycle for a shortened duration of 290 ms to eliminate over-turn drift.

5.6 I2C Bus Recovery Mechanism

Due to electrical noise generated by proximity to high-current inductive motor brushes, the synchronous `I2C1` serial lines are susceptible to bus lockups, where the slave device or noise spikes hold the serial data line (SDA) low indefinitely. To resolve this, the system executes an automated bit-banging bus recovery sequence immediately during power-on initialization before mounting the peripheral stack.

5.6.1 GPIO Register Overrides

The firmware enables the clock for the GPIOB peripheral bank and configures the hardware pins corresponding to the serial clock (SCL, `GPIO_PIN_8`) and serial data (SDA, `GPIO_PIN_9`) lines into manual general-purpose push-pull output modes (`GPIO_MODE_OUTPUT_PP`). This register override detaches the internal hardware `I2C` peripheral engine and grants direct bit-level control of the communication lines to the software layer.

5.6.2 Clock Pulse Injection Sequence

The firmware drives both lines high, holding them for 10 ms to establish a known baseline state. Next, the algorithm executes a recovery loop that cycles exactly nine times. During each iteration, the software toggles the clock line (SCL, `GPIO_PIN_8`) low for 5 ms, and then back high for 5 ms to step any hanging slave device through its internal register states:

```
for(int i = 0; i < 9; i++) {
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_8, GPIO_PIN_RESET);
    HAL_Delay(5);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_8, GPIO_PIN_SET);
    HAL_Delay(5);
}
```

Once complete, both lines are driven high for an additional 10 ms before the pins are reconfigured into their Alternate Function modes (`GPIO_AF4_I2C1`), allowing `HAL_I2C_Init()` to safely mount the serial interface without freezing the main execution path.

6 Working Principle

The operational architecture of the EdgeNav-X platform functions as a closed-loop perception-action engine. The vehicle translates raw environment inputs into deterministic spatial kinematics through five synchronized system stages: Power Distribution, Low-Level Compute Execution, Sensory Perception, Actuation Control, and Autonomous Spatial Kinematics.

6.1 Perception-Action Loop Mechanics

The vehicle operates on a continuous polling and execution sequence. The software cycle is designed to balance continuous forward progress with high-priority polling structures to guarantee real-time obstacle avoidance and precise trajectory corrections based on surface markings. The pipeline cycles through the following core operational phases:

1. **Power Distribution Framework:** High-current power originates from the main battery pack via a mechanical isolation toggle switch. The raw voltage bus splits into two parallel tracks: a direct high-current line feeding the L298N H-bridge motor driver stages, and a filtered step-down line processed by a DC-DC buck converter. The buck converter outputs a stabilized low-noise voltage rail that drives the internal logic of the STM32F446RE microcontroller, the IR proximity sensor, and the TCS34725 RGB color sensor.

2. **Continuous Cruise State (Forward Kinematics):** Upon passing the power-on reset (POR) and executing the I2C bus recovery routine, the STM32F446RE configuration layer sets the directional control pins (`IN1` through `IN4`) into forward configuration. The dual independent hardware timers (`TIM2` and `TIM3`) scale the pulse-width modulation output channels to a 450 duty cycle register depth, maintaining a uniform baseline forward velocity.

3. **Proximity Sensing Layer (Perception):** While traveling forward, the microcontroller continuously samples the digital input register tied to `GPIOA_PIN_8`. The IR proximity sensor projects an infrared beam ahead of the vehicle; when an obstacle falls within the detection range, the sensor output drops to a logic low state (`0`). The microcontroller captures this edge-triggered event, exits the low-speed cruise state, and executes a 40 ms software debounce confirmation.

4. **Active Braking and Stabilization State:** If the obstacle is verified, the HAL perception-action engine invokes an aggressive active braking routine. The GPIO pins reverse their directional states (`IN2` and `IN4` driven high) to create immediate reverse magnetic flux inside the motor coils, while the timers briefly punch high asymmetric compare registers (`CCR2 = 850`, `CCR3 = 750`) for 40 ms to counter left-side drift. Once forward velocity hits zero, the PWM lines drop to 0, and a 2-second mechanical settling delay is applied to prevent physical chassis sway from distorting subsequent optical readings.

5. **Chromatic Ingestion and Decision State:** The microcontroller initiates a 5-sample voting sequence over the 100 kHz I^2C1 bus. Every 110 ms, it reads the 16-bit raw registers (Clear, Red, Green, Blue) from the TCS34725 color sensor. These readings are mathematically normalized down to an 8-bit scale factor relative to the clear channel intensity:
$$C_{scaled} = \frac{\text{Color}_{Raw} \times 255}{\text{Clear\_Color}}$$
A majority vote counter determines the dominant surface color under the sensor. If a color accumulates ≥ 3 votes, it achieves an execution lock.

6. **Kinematic Execution Output (Action):** The verified color lock is matched to its assigned kinematic routing profile inside the navigation decision logic:

   - *White Lock:* Triggers a 180-degree pivot (`Robot_UTurn()`) by running both channels at a 460 duty cycle for 750 ms.
   - *Red Lock:* Forces a hard 5-second structural halt before re-entering cruise routines.
   - *Green Lock:* Executes a sharp right pivot turn (`Robot_Turn_Right()`) by setting a 160 duty cycle for an increased 410 ms window to neutralize structural under-steer.
   - *Blue Lock:* Executes a sharp left pivot turn (`Robot_Turn_Left()`) by setting a 160 duty cycle for a compressed 290 ms window to eliminate over-steer.

   Once the turn profile completes, the system invokes an initial 20–30 ms inertia kickstart pulse at an 850/950 duty cycle to clear static friction before dropping back to the baseline cruise state, closing the perception-action loop.

7 Challenges Faced and Solutions

During the integration and testing phases of the EdgeNav-X autonomous platform, several critical hardware-firmware bottlenecks emerged. These challenges spanned electrical noise interference, mechanical friction thresholds, sensor signal ambiguities, and driver voltage regulation anomalies, each requiring dedicated algorithmic solutions implemented within the low-level MCU firmware layer.

7.1 I2C Bus Lockups due to Inductive Motor Noise

Challenge: During initial multi-subsystem testing, the TCS34725 RGB color sensor would randomly stop responding over the `I2C1` bus, completely freezing the robot's primary control loop. Diagnostic testing with an oscilloscope revealed that the proximity of high-current inductive DC motor brushes generated significant electromagnetic interference (EMI). These voltage spikes caused the slave device to miss clock pulses, leaving its internal state machine out of sync and holding the Serial Data Line (`SDA`) low indefinitely. Because the standard `HAL_I2C` libraries rely on polling-based blocking loops, they would hang indefinitely while waiting for the bus to clear.

Solution: An automated bit-banging bus recovery sequence was implemented directly inside the power-on initialization block, executing before the master peripheral stack mounts. The firmware temporarily reconfigures `GPIOB_PIN_8` (`SCL`) and `GPIOB_PIN_9` (`SDA`) as manual push-pull outputs. The processor injects exactly nine clock pulses by toggling the manual `SCL` pin high and low at 5 ms intervals:

```
for(int i = 0; i < 9; i++) {
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_8, GPIO_PIN_RESET);
    HAL_Delay(5);
```

```
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_8, GPIO_PIN_SET);
    HAL_Delay(5);
}
```

This software-driven sequence forces the hanging slave device to step through its internal data register buffer and release the locked SDA line. Once the bus clears, the pins are restored to their alternate peripheral configurations (GPIO_AF4_I2C1) for seamless sensor communication.

7.2 Static Friction and Rotor Inertia at Low Duty Cycles

Challenge: To ensure accurate path tracking and color classification, the vehicle needed to travel at a slow, controlled cruise speed. However, when the hardware timers applied a stable, low-velocity duty cycle (CCR = 450), the TT geared motors failed to spin from a dead stop. The torque generated at low duty cycles was insufficient to overcome internal gearbox static friction and chassis inertia. Conversely, increasing the continuous duty cycle to guarantee an initial spin caused the robot to move too quickly, causing it to overshoot color track tokens.

Solution: The firmware introduces a brief, high-torque inertia override pulse within the locomotion API. Whenever the robot transitions from a static state to forward travel via Robot_Forward(), the compare registers are pulsed at a heavy duty cycle (CCR = 950) for exactly 25 ms. This instantaneous voltage surge breaks the static friction barrier and overcomes the rotor's initial inertia. The firmware then immediately drops the registers back to the stable cruise value of 450, maintaining the desired low-speed velocity profile without stalling.

7.3 Chromatic Overlap and Ambient Sensor Misclassification

Challenge: During field evaluation across varying floor surfaces, the TCS34725 optical sensor exhibited severe color classification ambiguities. Beneath the chassis, complex ambient shadowing and dark floor grain patterns compressed the normalized 8-bit dynamic ranges. Consequently, when the sensor crossed specific boundary paths, the scaled red component (r_{scaled}) and alternative color channels would generate nearly identical fractional metrics (e.g., matching within a narrow ±5 register variance margin). Under naive thresholding rules, this tiny variance triggered rapid, erratic state toggles between contradictory navigation routes, destabilizing the robot's state machine.

Solution: A robust color differentiation safety override was baked directly into the decision-making logic of the 5-sample voting engine. When processing the scaled chromaticity values, the firmware enforces a strict relational tie-breaker rule. If the mathematical delta between the target channel and a competing channel drops below a calibrated multi-sample threshold, the firmware flags the reading as a dangerous chromatic overlap. To guarantee predictable track tracking, the classification routine executes an explicit rejection branch:

```
if (abs(r_scaled - alternative_scaled) < CHROMA_THRESHOLD) {
```

```
    // Override ambiguous state and force fallback classification
    dominant_color = FALLBACK_PATH_COLOR;
}
```

By intentionally dropping the ambiguous red state and forcing a dedicated fallback path selection, the algorithm isolates the system from boundary-condition noise and guarantees smooth, monolithic steering executions.

7.4 Motor Asymmetry and Carbon Brush Mechanical Degradation

Challenge: The low-cost TT geared DC motors demonstrated non-linear physical operating behaviors over extended testing cycles. Internal construction tolerances and progressive mechanical wear on the internal carbon commuter brushes drastically altered the motor's current draw profile. This degradation introduced continuous drift along the linear cruise vectors. Applying symmetric timer compare values caused the chassis to veer uncontrollably to one side as the brushes degraded, destroying path-following stability.

Solution: To counteract this evolving mechanical drift, the software steering API incorporates independent left and right duty cycle bias parameters. Rather than relying on rigid, shared duty limits, the firmware implements distinct timing windows and asymmetrical core execution registers during localized directional adjustments. The right pivot configuration (`Robot_Turn_Right()`) utilizes an expanded execution window of 410 ms at a 160 duty register, while the left pivot loop (`Robot_Turn_Left()`) is tightly bounded to a compressed 290 ms burst. This decoupled framework allows developers to trim the runtime parameters independently, counterbalancing any ongoing mechanical degradation without altering the global control loops.

7.5 H-Bridge Voltage Drops and L298N Thermal Dissipation

Challenge: The L298N dual H-bridge motor driver exhibited noticeable voltage losses during operation due to its internal Darlington transistor architecture. Under continuous motor loading and frequent braking operations, a significant portion of the supply voltage was dissipated as heat within the driver rather than being delivered to the TT geared DC motors. This reduced the effective motor voltage and occasionally affected turning consistency and braking performance during extended test runs.

Solution: To compensate for the inherent voltage drop characteristics of the L298N, the motor control parameters were experimentally calibrated through multiple test iterations. Independent PWM control using TIM2 and TIM3 allowed fine adjustment of the left and right motor speeds to maintain stable motion despite variations in delivered voltage. Additionally, an active braking mechanism was implemented within the `Robot_Stop()` routine, where short-duration reverse PWM pulses were applied before power removal. This technique rapidly dissipated residual momentum, improved stopping accuracy, and ensured consistent navigation behavior without requiring hardware modifications to the motor driver.

7.6 False-Positive Proximity Triggers from Optical Noise

Challenge: The active-low IR obstacle detection sensor frequently suffered from brief voltage fluctuations. Vibrations from movement across uneven surfaces and changes in ambient lighting conditions caused the sensor line (`GPIOA_PIN_8`) to drop to logic `0` for fractions of a millisecond. In earlier iterations, these transient spikes were treated as real obstacles, triggering unnecessary active braking states and halting forward progress on an empty path.

Solution: A temporal software-filtering protocol was integrated directly into the sensor polling loop. When a logic low state is detected on the IR flag pin, the firmware halts execution and establishes a 40 ms debouncing confirmation window using `HAL_Delay(40)`. After this window expires, the firmware performs a second, discrete check on the pin register. If the state has returned to logic high, the event is filtered out as ambient noise, and the robot continues moving forward without interrupting its cruise state.

8 Results and Discussion

The EdgeNav-X autonomous navigation platform was successfully tested under various operating conditions. Experimental evaluation confirmed reliable integration of obstacle detection, color perception, autonomous decision making, and motion control within a single embedded platform.

The implemented I^2C bus recovery mechanism achieved stable communication with the TCS34725 color sensor, eliminating startup communication failures during testing. The 5-sample voting algorithm improved color classification reliability and maintained accurate detection under varying lighting conditions. Red, blue, green, and white color markers were successfully identified, allowing the robot to execute the corresponding navigation actions.

Independent PWM control using TIM2 and TIM3 significantly improved straight-line movement by compensating for motor performance variations. Calibration of turning parameters enabled accurate left turns, right turns, and U-turn maneuvers with minimal deviation from the intended path.

The active braking mechanism implemented through the L298N motor driver reduced stopping distance and improved positioning accuracy during color detection. Combined with the obstacle detection subsystem, the robot consistently stopped, identified colors, executed navigation commands, and resumed autonomous movement without human intervention.

Overall, the experimental results demonstrate that EdgeNav-X successfully integrates real-time perception, decision making, and motion control using an STM32 Cortex-M4 microcontroller platform.

9 GitHub Repository and Version Control

The complete source code, firmware implementation, hardware documentation, project images, and supporting resources associated with the EdgeNav-X autonomous navigation platform are publicly available via a cloud-managed repository. Development was managed using Git version control to track incremental modifications, handle rollback states during hardware testing phases, and maintain clean firmware release iterations.

9.1 Version Control Methodology

Git version control was utilized throughout the software lifecycle to enforce stability while introducing experimental low-level firmware changes. Code additions, particularly the disruptive register-level interventions such as the bit-banged I^2C recovery protocol and asymmetric motor tuning parameters, were systematically isolated using local development branches before being reviewed and merged into the primary production branch. Commit messages were structured to catalog precise updates to specific peripheral configurations, which minimized regression issues and allowed for quick rollbacks when dealing with tricky hardware variations.

9.2 Repository Access

The complete project space, including the underlying STM32CubeIDE workspace, circuit pinouts, system photographs, and structural report files, can be accessed through the following public link:

Link: https://github.com/Anorasharon/EdgeNav-X-STM32-Cortex-M4-Autonomous-Perception-Motion-Control-Platform-

This open-source repository serves as a comprehensive archive of the EdgeNav-X platform, allowing developers to inspect the raw Embedded C source files, evaluate the peripheral initialization structures, or replicate the sensory perception and dual-timer motor control subsystems.

10 Results

The EdgeNav-X autonomous navigation platform was successfully implemented and tested using an STM32F446RE microcontroller, TCS34725 RGB sensor, IR obstacle sensor, and L298N motor driver. The robot accurately detected red, green, blue, and white color markers and executed the corresponding navigation actions in real time.

The independent PWM control using TIM2 and TIM3 improved straight-line motion by compensating for motor performance variations. The obstacle detection subsystem reliably halted the robot when obstacles were detected and resumed operation after clearance.

The active braking mechanism reduced stopping distance and improved positioning accuracy during color detection. The implemented firmware successfully handled sensor acquisition, color classification, motor control, and navigation decisions without system instability.

Overall, the experimental results demonstrate that EdgeNav-X successfully achieves autonomous perception and motion control with reliable real-time performance.